

Module M214

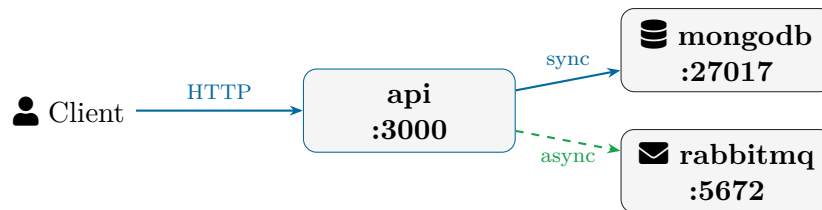
Application Cloud Native

Travaux Pratiques

Docker Compose — API + MongoDB + RabbitMQ

Durée : 1h30 **Niveau :** Intermédiaire

Pré-requis : Node.js, Docker Desktop, cours Docker Compose



Contents

1	Exercice 1 — Préparer le projet	2
2	Exercice 2 — Le fichier <code>.env</code>	2
3	Exercice 3 — L'API Node.js	3
4	Exercice 4 — Le Dockerfile de l'API	4
5	Exercice 5 — Le fichier <code>docker-compose.yml</code>	5
6	Exercice 6 — Démarrage et tests	6
7	Exercice 7 — Gestion du cycle de vie	7
8	Exercice 8 — Questions de synthèse	8
9	Exercice Bonus — Aller plus loin	9

i Consignes

- Ne copiez pas — rédigez vous-même chaque fichier et commande.
- Chaque exercice s'appuie sur le précédent : avancez dans l'ordre.
- En cas d'erreur, lisez le message dans le terminal avant de demander de l'aide.
- Consultez le cours pour les directives Compose, les URL de connexion et les commandes.

1 Exercice 1 — Préparer le projet

☰ Structure du projet

Créez le dossier `tp-compose` et la structure suivante. Pour chaque élément, indiquez son rôle dans la colonne de droite.

Fichier / Dossier	Rôle
<code>tp-compose/</code>	Dossier racine du projet
<code>tp-compose/api/</code>	
<code>tp-compose/api/app.js</code>	
<code>tp-compose/api/package.json</code>	
<code>tp-compose/api/Dockerfile</code>	
<code>tp-compose/.env</code>	
<code>tp-compose/docker-compose.yml</code>	

2 Exercice 2 — Le fichier `.env`

☰ Variables d'environnement

1. Créez le fichier `.env`. Pourquoi ne faut-il pas le commiter dans Git ?

2. Complétez le tableau ci-dessous avec les variables dont vous aurez besoin, en choisissant vos propres valeurs :

Variable	Valeur choisie
MONGO_ROOT_USER	
MONGO_ROOT_PASS	
RABBITMQ_USER	
RABBITMQ_PASS	

3 Exercice 3 — L'API Node.js

☰ Coder l'API avec Mongoose et amqplib

Placez-vous dans le dossier `api/`, initialisez un projet Node.js et installez les dépendances nécessaires.

Quels packages devez-vous installer ? _____

Schéma Mongoose à implémenter :

Champ	Type	Contraintes
nom	String	Obligatoire
prix	Number	Obligatoire, min 0
stock	Number	Valeur par défaut : 0

Routes à implémenter dans `app.js` :

Méth.	Route	Comportement
GET	/	Message de bienvenue + statut
GET	/api/produits	Retourner tous les produits depuis MongoDB
POST	/api/produits	Créer un produit ET publier un message dans RabbitMQ

⚠ URL de connexion dans Docker Compose

L'URL MongoDB et l'URL RabbitMQ doivent utiliser le **nom du service** déclaré dans `docker-compose.yml`, pas `localhost`. Ces URLs doivent être lues depuis des variables d'environnement.

Quelle URL utiliserez-vous pour MongoDB (format complet) ?

Quelle URL utiliserez-vous pour RabbitMQ ?

4 Exercice 4 — Le Dockerfile de l'API

☰ Ecrire le Dockerfile

Créez le fichier `api/Dockerfile`. Remplissez chaque instruction dans le tableau ci-dessous, puis recopiez-les dans le fichier.

#	Ce que l'instruction doit faire	Instruction à écrire
1	Image de base Node.js 20 (variante légère)	
2	Répertoire de travail <code>/app</code>	
3	Copier les fichiers <code>package*.json</code>	
4	Installer les dépendances (production)	
5	Copier tout le code source	
6	Déclarer le port 3000	
7	Lancer l'application	

5 Exercice 5 — Le fichier `docker-compose.yml`

☰ Planifier les services

Avant d'écrire le fichier, complétez ce tableau pour chaque service :

Service	Image ou build	Ports	Variables d'env.	Dépend de
mongodb		27017:27017	MONGO_INITDB_ROOT_USERNAME ..._PASSWORD	—
rabbitmq			RABBITMQ_DEFAULT_USER ..._PASS	—
api		3000:3000	PORT, MONGO_URI, RABBITMQ_URL	

Quels volumes nommés allez-vous déclarer ? Pourquoi ?

Quel nom donnerez-vous au réseau interne ?

☰ Rédiger le `docker-compose.yml`

Rédigez maintenant le fichier `docker-compose.yml` complet dans votre éditeur de code, en vous basant sur le tableau ci-dessus.

i Structure de base

- Version : '3.8'
- Ajoutez `restart: always` sur chaque service
- Déclarez les volumes nommés dans la section `volumes:` à la racine
- Tous les services sur le même réseau avec `driver: bridge`

 RabbitMQ — deux ports

L'image `rabbitmq:3-management` expose deux ports : **5672** (connexion AMQP des applications) et **15672** (interface web de management, accessible sur `http://localhost:15672`).

6 Exercice 6 — Démarrage et tests

☰ Lancer et tester l'architecture

1. Écrivez la commande pour construire les images et démarrer tous les services en arrière-plan. _____
2. Écrivez la commande pour vérifier l'état de tous les services. _____
3. Les trois services sont-ils **Up** ? ☐ Oui ☐ Non
4. Si l'API ne démarre pas, quelle commande vous permet de voir la cause de l'erreur ? _____
5. Écrivez la commande `curl` pour **créer un produit** via l'API (corps JSON à votre choix) : _____

6. Écrivez la commande pour **lister tous les produits** : _____
7. Ouvrez l'interface de management RabbitMQ dans le navigateur. Quelle URL utilisez-vous ? _____

Dans l'interface, vérifiez que la queue `produits` est visible et qu'un message y a bien été publié.

La queue est-elle visible ? ☐ Oui ☐ Non

7 Exercice 7 — Gestion du cycle de vie

☰ Commander le cycle de vie

1. Arrêtez tous les services **sans supprimer** les conteneurs ni les données.

2. Redémarrez les services arrêtés. _____

3. Vérifiez que les produits créés précédemment sont toujours dans MongoDB.
Sont-ils présents ? ☐ Oui ☐ Non

Quel mécanisme garantit cette persistance ?

4. Modifiez la route `GET /` de votre API pour y ajouter un champ **version**:
"2.0". Quelle commande utilisez-vous pour reconstruire **uniquement l'image**
de l'API ? _____

5. Écrivez la commande pour entrer dans le conteneur MongoDB et lancer un shell
interactif. _____

6. Effectuez un **nettoyage complet** : arrêtez et supprimez tous les conteneurs, le
réseau et **les volumes**. _____

Après ce nettoyage, que devient la base de données MongoDB ? Pourquoi ?

8 Exercice 8 — Questions de synthèse

Questions de synthèse —

Compréhension

1. Pourquoi les services utilisent-ils le nom du service (ex : `mongodb`) dans leur URL de connexion, plutôt que `localhost` ?

2. Quelle est la différence entre `docker compose stop` et `docker compose down` ? Et entre `docker compose down` et `docker compose down -v` ?

3. `depends_on`: - `mongodb` garantit-il que MongoDB est **prêt** à accepter des connexions quand l'API démarre ? Justifiez votre réponse et proposez une solution.

4. Quelle est la différence entre un **volume nommé** et un **bind mount** ? Dans quel cas utilisez-vous l'un ou l'autre ?

5. Dans l'exercice, le service `api` publie un message dans RabbitMQ lors de la création d'un produit. Qui lira ce message ? Quel est l'intérêt de cette communication **asynchrone** ?

9 Exercice Bonus — Aller plus loin

☰ Bonus

1. Ajoutez une route **DELETE** `/api/produits/:id` dans l'API qui supprime un produit de MongoDB **et** publie un événement `PRODUIT_SUPPRIME` dans RabbitMQ.
2. Ajoutez une route **GET** `/api/produits?stock=0` qui filtre et retourne uniquement les produits en rupture de stock (`stock = 0`).
3. Ajoutez un **healthcheck** au service `api` dans `docker-compose.yml` qui interroge la route **GET** `/` toutes les 30 secondes.
Quelle commande vérifie ensuite l'état de ce healthcheck ?

4. Ajoutez un service **consommateur** dans le Compose : un second service Node.js qui s'abonne à la queue `produits` et affiche dans ses logs chaque message reçu.